

## UNIT 6# Basic I/O, Memory R/W and Interrupt Operations

---

Memory read/write, Input/output read/write operation in 8085 microprocessor based system

A memory unit is a collection of storage cells together with associated circuits needed to transfer information in out of storage. The memory stores binary information in groups of bits called words. A memory is a group of 1's and 0's and may represent a number, an instruction code one or more alphanumeric characters or any other binary-coded information. A group of 8 bits is called a byte. The capacity of memories in commercial computers is usually stated as the total number of bytes that can be stored. For 8 bit micro-computer system memory word and memory byte are the same. A memory word is identified by an address. The 8 bit microprocessor uses 16 bit address to access memory word. This provides a maximum of  $2^{16} = 65536$  memory addresses ranging from 0000H to FFFF H. Thus , the memory capacity for this Micro-computer system is 64K.

Memory can be classified as non-volatile memory or volatile memory. Non-volatile memory retains the stored data even when there is no power. On the other hand volatile memory losses its contents when the power is removed. Few examples are:

Non volatile: ROM, PROM, EPROM, Floppy disk, hard disk etc.

Volatile: RAM (static dynamic), CCD (Charge coupled devices etc).

In broad sense, a microcomputer's memory system can be divided into three groups;

- i. Processor memory
- ii. Primary memory
- iii. Secondary memory.

**Processor Memory:** Processor memory refers to a set of microprocessor registers. They are used to hold temporary results when a computation is in progress Although use of such registers enhances the execution speed, the cost involved in the approach forces a micro-computer designer to include only a few registers include the processor. In 8085 we have registers like A, B, C, D, E, H, L, SP, PC etc to store data temporarily.

**Primary Memory:**

It is the storage area where all programs are executed. The microprocessor can directly access only those items that are stored in the primary memory. Hence,

all programs and data must be within the primary memory prior to execution. Usually, the size of the primary memory is much larger than that of processor memory and its operating speed is much slower than processor's registers. Primary memories can be divided into two main groups:

- 1. Read only memory (ROM)
- 2. Random Access memory. (RAM)

## UNIT 6# Basic I/O, Memory R/W and Interrupt Operations

---

### RAM

It is used primarily for information that is likely to be altered, such as writing programs or receiving data. This memory is volatile. Two types of RAM are available.

**Static RAM:** This memory is made up of flip flops; and stores the bits as voltage. Each memory cell requires six transistors. This memory is more expensive and power consuming than dynamic memory. It is called 'static' because the information doesn't need a constant update. These memories are commonly used for cache memory. This types of memory is very fast with access time is 15 to 30 nanoseconds but is physically bulky.

**Dynamic RAM (DRAM) :** Dynamic random-access memory is an improvement over the expensive and bulky SRAM. DRAM uses a different approach to store data. Information is stored as charges in a very small capacitor. If a charge exists in a capacitor, it's interpreted as 1. The absence of a charge will be interpreted as 0. Because DRAM uses capacitors there is a chance of leakage of charge. Thus it needs to use a constant refresh signal to keep the information in the memory (every few millisecond). DRAM technology allows several memory units, called cells to be packed at very high density. Therefore, these chips can hold very large amount of information. Most PCs today use DRAM. Access time for DRAM is 80 nanoseconds or more, slower than SRAM, and two or three times faster than ROM

### Read only memory (ROM):

ROM is a non volatile memory and can be read only. It is used to store data and programs that are not to be altered. Among other things ROM is needed for storing an initial program called boot strap loader. The bootstrap loader is a program whose function is to start the computer software operating when power is turned on. Since RAM is volatile, its contents are destroyed when power is turned off. The contents of ROM remain unaltered after power is turned off and on again. The startup of a computer consists of turning the power on and starting the execution of an initial program. Thus when power is turned on, the hardware of the computer sets the program counter to the first address of the bootstrap loader. The bootstrap program loads a portion of the operation of the operating system from disk to main memory and control is then transferred to the operating system, which prepares the computer for general use.ROMs can be classified as below:

**Masked ROM:** They are permanent ROM recorded by masking. Generally manufacturers use this process to produce ROM in large numbers.

**PROM:** These are un-programmed ROM. The fuses on the ROM are not burned. A programmer can program this ROM according to his needs. The the information stored is permanent.

**EPROM:** These ROM can be reprogrammed and erased. Two types of such EPROM are available.

## UNIT 6# Basic I/O, Memory R/W and Interrupt Operations

**UV-EPROM** : The memory of such ROM can be erased by exposing the chip via a lid or window on the chip to ultraviolet light. The erase time generally varies between 10 to 30 minutes. The EPROM can be programmed by inserting the chip into a socket of the PROM programmer and providing proper addresses. The programming time varies from 1 to 2 minutes.

### EEPROM

This does not require U-V rays to be erased. It can be completely erased or have certain bytes changed, using electrical pulses. Writing to EEPROM is slower than writing to RAM, so it can not be used in high speed circuits.

### FLASH MEMORY:

This is a modified EEPROM. The difference is the erasure procedure. EEPROM can be erased at a register level, but flash memory must be erased either in its entirety or at the sector (block) level.

**Secondary Memory:** Secondary memory are storage devices. These devices have high data holding capacity. They store programs that are not frequently used by the processor. They are slow and have larger size. Some of the examples are: Hard disk, Floppy disk, Magnetic Tapes, CD, DVD etc.

### Basic concept in memory interfacing:

The primary function of memory interfacing is that two microprocessor should be able to read from and write into a given register of a memory chip. To perform this operation the micro processor should.

1. Be able to select the chip
2. Identify the register.
3. Enable the appropriate buffer.

For interfacing the chip with 8085  $\mu$ p we just need an additional decoder to select the chip.

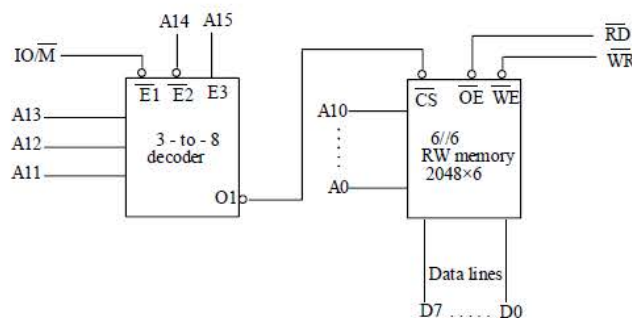


Fig. Interfacing R/W memory



## UNIT 6# Basic I/O, Memory R/W and Interrupt Operations

---

### Address decoding for memory interface

A microcomputer has microprocessor, memory & I/O devices attached. A processor communicates with all parts interconnected in the system through common address and data bus. Hence only one device can transmit data through the bus at a time & others can only receive data. Now, to ensure that the proper device gets addressed at proper time, the method of address decoding is used.

In this process memory blocks & I/O units are assigned a specific address. The address is determined by the way the device is connected to processor. The signal used to drive the device is called chip select. Since only one chip select is activated at a time only one block of memory or I/O device is activated. The purpose of address decoding circuit is to ensure that the **Chip select** signals to other devices are not activated. There are two methods for mapping address of these devices. They are: **I/O mapped I/O** and **Memory mapped I/O**

In **I/O mapped I/O** method the I/O devices are addressed with 8-bit address. It means that the Chip Select signals for the devices are derived by using the 8 address lines. In 8085 microprocessor, the general procedure involves the use of lower 8 bit address lines for deriving the Chip Select signal. Thus only 256 devices or addresses can be mapped. The instruction used for input and output are IN and OUT.

In **memory mapped memory I/O** mode the I/O devices are addressed with 16 bit data. It means that in this mode the Chip Select signals for the devices are derived using all 16 address lines from the processor. Thus a total of 64 KB can be addressed.

Usually in microcomputers based on 8085 processor the memories like RAM, ROM, EPROM etc uses memory mapping whereas the devices like 8255A, 8251A, input/output latches etc use I/O mapping. Now depending on the addresses that are allocated to the device the address decoding can be categorized as:

1. **Unique address decoding (Absolute address decoding):** If all of the address lines available on that mapping mode are used for address decoding, then that decoding is called unique address decoding. Thus, in memory mapped memory I/O all 16 lines are used and in I/O mapped I/O method all 8 lines are used for address decoding.
2. **Non-Unique address decoding (Partial Address Decoding):** If all of the address lines available on that mode are not used address decoding, then that decoding is called non-unique address decoding. Such practice of address decoding is simple but rather conflicting as the device is activated for a wide range of addresses.

### Direct Memory Access (DMA)

The transfer of data between a fast storage device such as magnetic disk and memory is often limited by the speed of the CPU. Removing the CPU from the path and letting the peripheral device manage the memory buses directly would improve the speed of transfer . This transfer technique is called direct memory access(DMA).

## UNIT 6# Basic I/O, Memory R/W and Interrupt Operations

---

DMA is used in large blocks of data transfer at a high speed to or from high speed devices, magnetic drums, disks, tapes, etc. We use DMA controller which is interface that provides I/O transfer of data directly to and from the memory and the I/O device. CPU initializes the DMA controller by sending a memory address and the number of words to be transferred. Actual transfer of data is done directly between the device and memory through DMA controller freeing CPU for other tasks.

In the 8085, DMA (Direct Memory Access) is controlled with **HOLD** and **HLDA**. The HOLD signal is a request to release the bus. The HLDA signal is the 8085's acknowledgement of that request. HLDA means that the 8085 will release the bus in one half clock cycle, i.e. at the end of T3. The 8085 will remain in that hold state until HOLD is released, at which point it will take control of the buses again. The HOLD'ing device has complete control and can access any memory or I/O. Often, the 8237 DMA controller is used to provide sequencing of the operation. The 8237 DMA controller interfaces between up to four peripheral devices and the 8085. It provides an address register for each device so that the device does not need to do so. The device only needs to indicate that a byte of data is available, or is required, and the 8237 will take care of storing or fetching the byte.

The 8237A Multimode Direct Memory Access (DMA) Controller is a peripheral interface circuit for microprocessor systems. It is designed to improve system performance by allowing external devices to directly transfer information from the system memory. Memory-to-memory transfer capability is also provided. The 8237A offers a wide variety of programmable control features to enhance data throughput and system optimization and to allow dynamic reconfiguration under program control.

A DMA controller is designed to service one or more I/O or mass storage interfaces and each interface is connected to the controller by a set of conductors. A portion of a DMA controller for servicing a single interface is called a channel.

Each channel will have

- a) Control Register
- b) Status Register
- c) Address Register
- d) Byte Count Register

Actually, the 8237 includes Control and status register and four channels. Each channel contains a

- 1) mode register
- 2) current address register
- 3) base address register
- 4) current byte counter
- 5) base byte counter

## UNIT 6# Basic I/O, Memory R/W and Interrupt Operations

---

- 6) request flag and
- 7) mask flag

### The DMA interface

The DMA adds one more level of complexity to the I/O interface because a DMA controller has independent access to memory. One set of wires (bus) can carry at most one transaction at a time. If the DMA and a microprocessor share the signal wire to memory there must be a mechanism to arbitrate which shall have access to memory when both attempt to at the same time.

### Functional behavior of a DMA transaction

1. The processor transmits the following information to a DMA controller:
  - (a) beginning address in memory
  - (b) block length (number of words to transfer)
  - (c) direction (memory-to-device or device-to-memory)
  - (d) port ID
  - (e) end of block action (interrupt request or no interrupt request).
2. The processor returns to other activities while the DMA controller starts the data transfer.
3. Each time the DMA controller accesses memory, it synchronises this memory request with an idle period of the processor--to do this the possibilities are:
  - (a) force an immediate disabling of the processor, or
  - (b) request a halt of the processor, and await an acknowledgement, or
  - (c) time the DMA access to a clock interval or status signal of the processor that signals an idle cycle.
4. When the DMA controller accesses an I/O port or memory, it uses the same functional control signals as used by the processor. I/O port activity can be performed on dedicated lines that do not have to be synchronised with the processor.
5. At the completion of the block transfer, the DMA controller raises an interrupt request if the interrupts are "armed" and otherwise indicates completion in its status register.
6. The processor recognises I/O completion (either by interrupt or by reading the status register); thereafter the activity between the processor and the DMA controller follows the normal post-completion activity of any I/O port.



## UNIT 6# Basic I/O, Memory R/W and Interrupt Operations

---

This shows that the controller is treated as a standard port before and after block transfer and during transfer the DMA must be able to synchronise with the processor. The controller improves performance especially with a built-in program for moving a stream of data between memory and an I/O port--thereby not requiring to access the instruction from memory and executing them one by one. Some elementary actions can be performed in parallel instead of sequentially when implemented with software in the processor. For example, the controller decrements a counter each time it moves a datum. The controller can overlap the subtraction with memory access and avoid the time penalty for the arithmetic instruction. Because of the ability to achieve higher performance for block transfers, the DMA controller is used most frequently for high speed I/O, especially disk. Fast disks move blocks of data at speeds much greater than any program can control and therefore must be interfaced to computers through DMA controllers.

The four possible modes are

1) Single Transfer mode or cycle stealing mode (01)

After each transfer the controller will release the bus to the processor for at least one bus cycle, but will immediately begin testing for DREQ (DMA Request) inputs and proceed to steal another cycle as soon as a DREQ line becomes active.

### Cycle Stealing

While DMA I/O takes place, CPU is also executing instructions. When DMA Controller and CPU both access Memory, Memory Access Conflict arises. This is managed by Memory Bus controller. It coordinates the activities of all devices requesting memory access using priority system. Memory accesses by CPU and DMA Controller are interwoven, with the top priority given to DMA Controller. Cycle stealing is the mode in which DMA controller take over the bus for each byte of data to be transferred and then return control to the CPU. Since the processor originates most memory access cycles, it is often stated that DMA steals memory cycles from the processor. CPU is usually much faster than I/O(DMA), thus CPU uses the most of the memory cycles, DMA Controller steals the memory cycles from CPU. For those stolen cycles, CPU remains idle.

2) Block Transfer mode or burst transfer mode (10)

The bus control is not released by the DMA until the entire block of data has been transferred. In Block Transfer mode the device is activated by DREQ to continue making transfers during the service until word count goes to end count, or an external End of Process (EOP) is encountered. An autoinitialization will occur at the end of the service if the channel has been programmed for it.

3) Demand Transfer Mode (00)

This mode is similar to the block mode except that DREQ is tested after each transfer. If DREQ is inactive, transfers are suspended until DREQ once again becomes active, at

## UNIT 6# Basic I/O, Memory R/W and Interrupt Operations

---

which time the block transfer continues from the point at which it was suspended. This allows the interface to stop the transfer in the event that its device cannot keep up. This is due to memory latency or slow speed of the interface. Demand Transfer mode the device is programmed to continue making transfers until a word count ends or external end of process is encountered or until DREQ goes inactive. Thus transfers may continue until the I/O device has exhausted its data capacity. After the I/O device has had a chance to catch up, the DMA service is re-established by means of a DREQ. During the time between services when the microprocessor is allowed to operate, the intermediate values of address and word count are stored in the 8237A Current Address and Current Word Count registers. Only an EOP can cause an Autoinitialize at the end of the service. EOP is generated either by TC or by an external signal.

### 4) Cascade Mode(11)

In this mode 8237's are cascaded. This mode is used to cascade more than one 8237A together for simple system expansion. The HRQ and HLDA signals from the additional 8237A are connected to the DREQ and DACK signals of a channel of the initial 8237A. This allows the DMA requests of the additional device to propagate through the priority network circuitry of the preceding device. The priority chain is preserved and the new device must wait for its turn to acknowledge requests. Since the cascade channel of the initial 8237A is used only for prioritizing the additional device, it does not output any address or control signals of its own. These could conflict with the outputs of the active channel in the added device. The 8237A will respond to DREQ and DACK but all other outputs except HRQ will be disabled. The ready input is ignored.

## INTERRUPT

### Introduction

An Interrupt is an event that causes the processor to suspend its present task and transfer the present task and control to new program called ISR(Interrupt Service Routine ). Most MPU allow normal program execution to be interrupted by some external signal or by a special instruction in the program. In response to an interrupt, the MPU stops executing its current program and calls a procedure which services the interrupt. The actual code that is invoked when an interrupt occurs is called the **Interrupt Service Routine (ISR)**.

Interrupts and exceptions alter the normal program flow in order to handle external events, report errors or exceptional conditions. The difference between interrupts and exceptions is that interrupts are used to handle asynchronous external events while exceptions handle instruction faults. Although a program can generate a software interrupt via an INT N instruction, the processor treats software interrupts as exceptions.

An interrupt is a signal that a peripheral board sends to the central processor in order to request attention. In response to an interrupt, the processor stops what it is currently doing and executes a service routine. When the execution of the service routine is terminated, the original process may resume its previous operation.



## UNIT 6# Basic I/O, Memory R/W and Interrupt Operations

---

The interrupt is initiated by an external device and is asynchronous, meaning that it can be initiated at any time without reference to system clock. However, the response to an interrupt request is directed or controlled by the microprocessor

Interrupts are primarily issued on:

- Initiation of I/O operation.
- Completion of I/O operation.
- Occurrence of hardware or software errors

Process of interrupt operation:

1. The I/O unit issues an interrupt signal to the processor. An interrupt signal from I/O is the request for exchange of data with the processor.
2. The processor finishes execution of the current instruction before responding to the interrupt.
3. The processor tests for an interrupt, determines that there is one, and sends an acknowledgement signal to the device that issued the interrupt. After receiving this acknowledgement, the device retains its interrupt signal.
4. The processor now begins to transfer the control to the routine which serves the interrupt request from the device. This routine is called 'Interrupt service routine' and it resides at a specified memory location. For this process, the CPU needs to save information needed to resume the current program at the point of interrupt. The minimum information required is
  - i) the status of the processor, which contained by the processor status word (PSW) and
  - ii) the location of the next instruction to be executed which is contained by the program counter (PC), these all are pushed onto the stack.
5. The processor then loads the program counter with the entry location of the interrupt service routine that will respond to this interrupt. Once the program counter has been loaded, the control is transferred to the interrupt handler program.
6. The fundamental requirement of the interrupt service routine is that it should begin by saving the contents of all the registers on the stack (as state of the main program should be safe). Suppose the user program is interrupted after the instruction at location N. The contents of all the registers plus the address of the next instruction are saved on the stack. The stack pointer is updated and the program counter is updated to point to the beginning of the interrupt service routine.
7. The interrupt handler now proceeds to process the interrupt. This will include an examination of status information relating to the I/O operation or the other event that caused an interrupt. It may also involve sending additional commands or acknowledgement to the I/O unit.
8. When interrupt processing is complete the saved register's value (of the main program) are retrieved from the stack and restored to the register.

## UNIT 6# Basic I/O, Memory R/W and Interrupt Operations

---

9. The final function is to restore the PSW and program counter values from the stack. As a result the next instruction to be executed will be from the previously interrupted main program.

### Sources of Interrupts

There are three sources of an interrupts and they are as follows.

#### 1. PROCESSOR INTERRUPT

These are internal interrupts and are generated by the processor itself, usually in response to an error condition. For example a type 0 interrupt occurs when attempt to divide by zero. Processor interrupt arise from illegal or erroneous use of an instruction or data. Cause of this interrupt may be: register overflow, attempt to divide by zero, an invalid operation code, stack overflow etc. These error conditions usually occur as a result of premature termination of the instruction execution. These are even termed as exceptions.

#### 2. SOFTWARE INTERRUPT

These are special instructions that trigger an interrupt response to processor. Hardware and Processor interrupts are initiated from signal that occurs in the hardware of the CPU. A software interrupt is initiated by executing an instruction. Software interrupt is a special call instruction that behaves like an interrupt rather than a subroutine call. It can be used by the programmer to initiate an interrupt procedure at any desired point in the program. In 8085 the instruction like RST0, RST1, RST2, and RST3....etc. causes a software interrupt. The general form of the software interrupt instruction is INT 10H as in case of 8086.

#### 3. HARDWARE INTERRUPT

Hardware interrupts are External interrupt requests initiated by external hardware. 8086 has 2 pins reserved for this purpose (NMI and INTR). Interrupts are initiated via the microprocessor's interrupt pins by external devices such I/O devices, timing device, circuit monitoring the power supply etc. External interrupts depends on external conditions that are independent of the program being executed at the time. Causes of these interrupts may be; I/O device requesting transfer of data, I/O device finished transfer of data, elapsed time of an event, or power failure. Timeout interrupt may result from a program that is an endless loop and thus exceeded its time allocation. Power failure interrupt may have as its service routine a program that transfers the complete state of the CPU into a non-destructive memory in few milliseconds before power ceases. External interrupts can be further divided into two types:

1. Maskable interrupt.
2. Non-maskable interrupt.

**Maskable interrupt:** Those interrupts which can be blocked by user using instructions are called maskable interrupt. A maskable interrupt is one which can be enabled or disabled by executing instructions such as EI (enable interrupts) and DI (Disable interrupt). If the microprocessor's 'interrupt enable flip flop' is disabled, it ignores a maskable interrupt. In 8085,



## UNIT 6# Basic I/O, Memory R/W and Interrupt Operations

---

the 1 byte instruction EI sets the interrupt enable flip flop and enables the interrupt process. Similarly the 1 byte instruction DI resets the interrupt enable flip flop and disables the interrupt process. In 8086, **INTR** is a maskable hardware interrupt. The interrupt can be enabled/disabled using STI/CLI (STI-> Enable INTR Input and CLI-> Disable INTR Input) instructions or using more complicated method of updating the FLAGS register with the help of the POPF instruction. No maskable interrupts are recognized by the processor when the interrupt is disabled.

**Non maskable interrupt:** This type of interrupt cannot be enabled or disabled by instructions i.e cannot be blocked by instructions. This type has higher priority over the maskable interrupt. This means that if both the maskable and non maskable interrupts are activated at the same time, then the processor will service the non-maskable interrupt first. In 8085 TRAP is an example of non maskable interrupt. In 8086, NMI is a non-maskable interrupt. Another common use of this interrupt is to save program data in case of a system power failure or to deal with some other catastrophic failure conditions. Some external circuitry detects when the AC power to the system fails and sends an Interrupt signal to the NMI.

### INTERRUPT PRIORITY

Data transfer between the CPU and an I/O device is initiated by the CPU. However, the CPU cannot start the transfer unless the device is ready to communicate with the CPU. The readiness of the device can be determined from an interrupt signal. The CPU responds to the interrupt request by storing the return address from PC into a memory stack and then the program branches to a service routine that processes the required transfer. In micro-computer a number of I/O devices are attached to the processor, with each device being able to originate an interrupt request. The first task of the interrupt system is to indemnify the source of the interrupt. There is also the possibility that several sources will request service simultaneously. In this case the system must also decide which device to service first.

An interrupt priority is a system that establishes a priority over the various sources to determine which condition is to be serviced first when two or more requests arrive simultaneously. The system may also determine which conditions are permitted to interrupt the computer while another interrupt is being serviced. Higher-priority interrupt levels are assigned to requests which, if delayed or interrupted, could have serious consequences. Devices with higher speed transfers such as magnetic disks are given high priority, and slow devices such as keyboards receive low priority. When two devices interrupt the processor at the same time, the processor services the device with the higher priority first.

### Multiple Interrupts

If more than one device is connected to the interrupt line, the processor needs to know to which device service routine it should branch to. The identification of the device requesting service can be done in either hardware or software, or a combination of both. The three main methods are:

1. Polled Interrupts



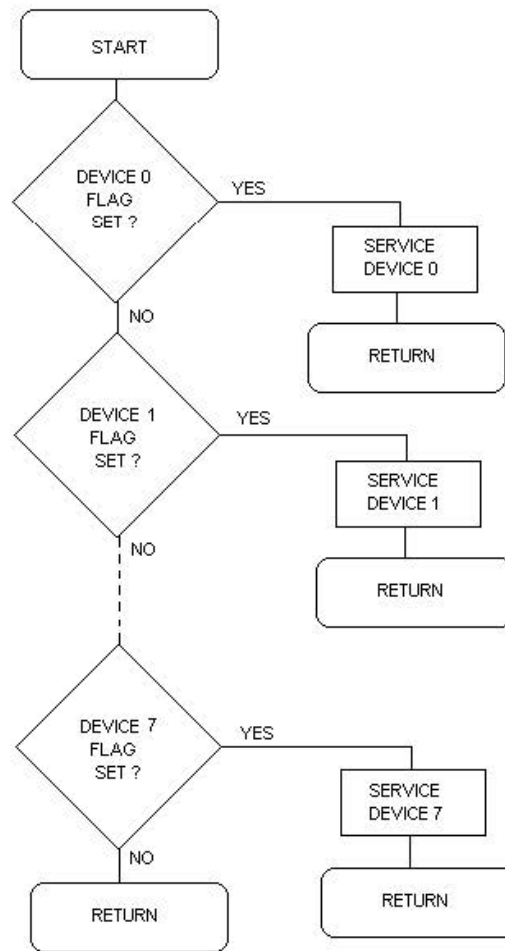
### 2. Vectored Interrupts

#### 1. Polled Interrupt:

Polled interrupt are handled using software and are therefore slower compared to vectored (hardware) interrupts. In this method there is one common branch address for all interrupts. The program that takes care of interrupts begins at the branch address and polls the interrupts sources in sequence. The order in which they are tested determines the priority of each interrupt. The highest priority source is tested first, and if its interrupt signal is on, control branches to a service routine for this source. Otherwise the next lower priority source is tested, and so on. Thus, the initial service routine for all interrupts consists of a program that tests the interrupt sources in sequence and branches to one of many possible service routines. Polled interrupts are very simple. But or large number of devices, the time required to poll each device may exceed the to service the device. In such case, the faster mechanism called Daisy chained interrupt ( Hardware Polled ) is used.

#### Software Polling

A software routine is used to identify the device requesting service. A simple polling technique is used, each device is checked to see if it was the one needing service. Having identified the device, the processor then branches to the appropriate interrupt-handling-routine address for the given device. The order in which the devices appear in the polling sequence determines their priority. Polling is the most common and simplest method of I/O control. It requires no special hardware and all I/O transfers are controlled by the CPU program. Polling is a synchronous mechanism, by which devices are serviced in sequential order.



SOFTWARE POLLING FLOWCHART

### Disadvantages :

- 1) It is wasteful of the processors time, as it needlessly checks the status of all devices all the time
- 2) It is inherently slow, as it checks the status of all I/O devices before it comes back to check any given one again,
- 3) When fast devices are connected to a system, polling may simply not be fast enough to satisfy the minimum service requirements,
- 4) Priority of the device is determined by the order in the polling loop, but it is possible to change it via software.

### 2. Vectored Interrupts:

This is hardware concept of handling the multiple interrupts. In this technique, the devices are connected to a priority controller chip for setting up the priority system. Here the device with the highest priority is programmed in the chip in the first position, followed by lower priority devices.



## UNIT 6# Basic I/O, Memory R/W and Interrupt Operations

---

Suppose that one or more devices interrupt the processor at a time. In response, the processor saves its current status and then generates an interrupt acknowledge (INTA) signal to the highest priority device. Once accepted, the device provides a means to the processor for finding the interrupt address vector using external hardware. Usually the requesting device responds by placing a word on the data lines. With the help of hardware it generates interrupts vector address. This word is referred to as vector, which the processor used as a pointer to the appropriate device service routine. This avoids the need to execute a general interrupt service routine first. So this technique is also referred to as vectored interrupts.

This is the fastest system. The focus is placed on the requesting device to request the interrupt, and identify itself. The identity could be a branching address for the desired interrupt-handling routine. If the device just supplies an identification number, this can be used in conjunction with a lookup table to determine the address of the required service routine. Response time is best when the device requesting service also supplies a branching address. Some Chips like 8259 PIC are used for this purpose.

The disadvantages are:

- 1) extra chip required,
- 2) resultant increases in cost,
- 3) more board space and power consumption,
- 4) fixed priority in hardware.

### VECTORED VS POLLED INTERRUPT

#### 1. VECTORED INTERRUPT

In vectored interrupt, the interrupt signal includes the identity of the device sending the interrupt signal. In a computer, a vectored interrupt is an I/O interrupt that tells the part of the computer that handles I/O interrupts at the hardware level that a request for attention from an I/O device has been received and also identifies the device that sent the request. In a vectored interrupt, the source that interrupts supplies the branch information (Interrupt Vector) to the CPU.

#### 2. POLLED INTERRUPT

It is an interrupt handler that requires the interrupt handler poll or checks a signal from each device in order to find out which one sent the interrupt request. In a computer, a polled interrupt is a specific type of I/O interrupt that notifies that part of the computer containing the I/O interface that a device is ready to be read or otherwise handled but does not indicate which device. The interrupt controller must poll each device to determine which one made the request. In a non-vectored interrupt, the branch address is assigned to a fixed location in the memory.

### 8259 PIC

The processor 8086 had two hardware interrupt pins. Out of these two interrupt pins one pins were allotted fixed vector address i.e NMI was assigned type 2 interrupt but the pin INTR was not allotted any vector address, rather than external device was supposed to hand over the type of interrupt to the CPU. The processor then gets this type and derives the interrupt vector address from that. Consider an application, where a number of IO devices are connected with CPU desire to transfer data using interrupt driven mode. In these case, more number of interrupt pins are required than available in a CPU. Moreover, in these multiple interrupt systems, the processor will have to take care of priorities for the interrupt, simultaneously occurring at the interrupt request pins. The 8086 has only two interrupt inputs, NMI and INTR.

To overcome all these difficulties, we require a programmable interrupt controller which is able to handle a number of interrupt at a time. This relieves the processor from this entire task. The 8259 was designed to operate only with 8 bits microprocessors like 8085.A modified version 8259A is compatible with 8 bit as well as 16 bits processor .

**1.1 Interrupt Request Register (IRR):** The interrupts at IRQ input lines are handled by Interrupt Request internally. IRR stores all the interrupt request in it in order to serve them one by one on the priority basis.

**1.2 In-Service Register (ISR):** This stores all the interrupt requests those are being served, i.e. ISR keeps a track of the requests being served.

**1.3 Priority Resolver :** This unit determines the priorities of the interrupt requests appearing simultaneously. The highest priority is selected and stored into the corresponding bit of ISR during INTA pulse. The IR0 has the highest priority while the IR7 has the lowest one, normally in fixed priority mode. The priorities however may be altered by programming the 8259A in rotating priority mode.

**1.4 Interrupt Mask Register (IMR) :** This register stores the bits required to mask the interrupt inputs. IMR operates on IRR at the direction of the Priority Resolver.

**1.5 Interrupt Control Logic:** This block manages the interrupt and interrupt acknowledge signals to be sent to the CPU for serving one of the eight interrupt requests. This also accepts the interrupt acknowledge (INTA) signal from CPU that causes the 8259A to release vector address on to the data bus.

**1.6 Data Bus Buffer :** This tristate bidirectional buffer interfaces internal 8259A bus to the microprocessor system data bus. Control words, status and vector information pass through data buffer during read or write operations.

**1.7 Read/Write Control Logic:** This circuit accepts and decodes commands from the CPU. This block also allows the status of the 8259A to be transferred on to the data bus.

**1.8 Cascade Buffer/Comparator:** This block stores and compares the ID's all the 8259A used in system. The three I/O pins outputs when the 8259A is used as a master. The same pins act as inputs when the 8259A is in slave mode. The 8259A in master mode sends the ID of the interrupting slave device on these lines. The slave thus selected, will send its preprogrammed vector address on the data bus during the next INTA pulse.

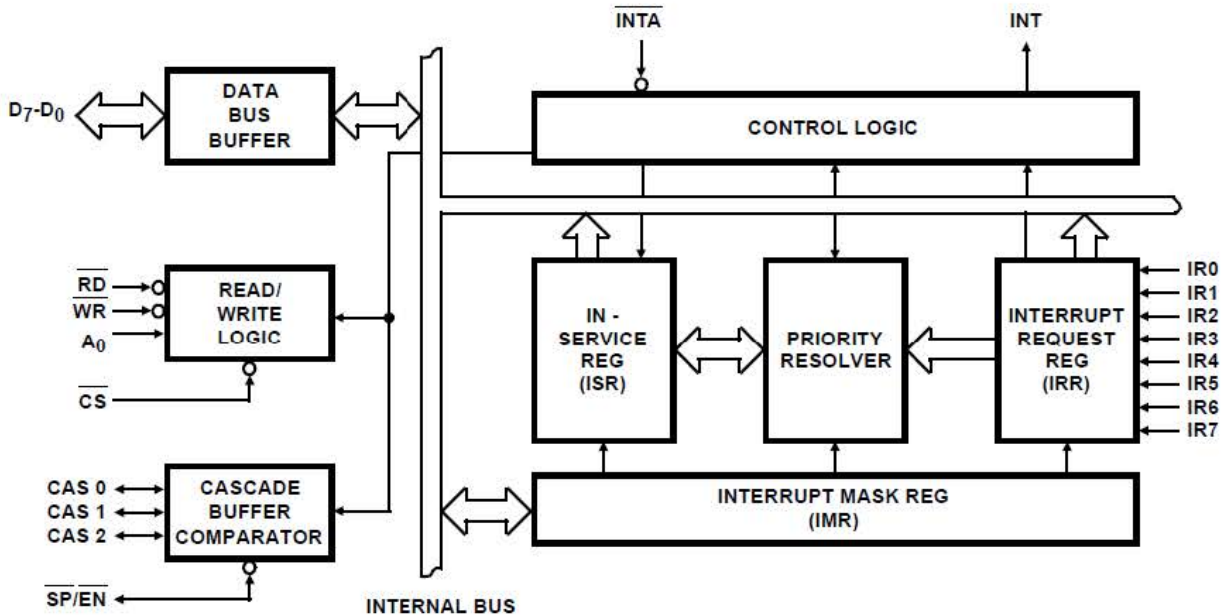


Figure : Block diagram of 8259 A

### Operation of 8259

It funnels the interrupt signals into a single interrupt input on the processor. The process is as follows :

1. **INTR** Interrupt Request comes to 8086 at **INTR** pin .
2. 8086 will send out Interrupt Ack pulse on its **INTA** pin to the **INTA** pin of the 8259A PIC.
3. The **INTA** pulse tells the 8259A to send the desired interrupt type on the data bus.
4. Multiply the interrupt type it receives from the 8259A by 4 to produce an address in the Interrupt Vector Table.
5. Push the flags on the stack.
6. Clear **IF** and **TF**
7. Push the return address on the stack.
8. Get the starting address for **ISR** (**CS: IP**).
9. Execute Interrupt Service Routine.
10. Return from Interrupt Service Routine.

### Operating Modes of 8259

The different modes of operation of 8259A can be programmed by setting or resting the appropriate bits of the **ICW** (Instruction Command Word ) or **OCW** ( Operation Command Word ).The different modes of operation of 8259A

- Fully Nested Mode
- End of Interrupt (EOI)
- Automatic Rotation
- Automatic EOI Mode
- Specific Rotation
- Specific Mask Mode



## UNIT 6# Basic I/O, Memory R/W and Interrupt Operations

---

- Edge and Level Triggered Mode
- Reading 8259 Status
- Poll Command
- Special Fully Nested Mode
- Buffered Mode
- Cascade Mode

### **Fully Nested Mode :**

This is the default mode of operation of 8259A. IR0 has the highest priority and IR7 has the lowest one. When interrupt request are noticed, the highest priority request amongst them is determined and the vector is placed on the data bus. The corresponding bit of ISR is set and remains set till the microprocessor issues an EOI command just before returning from the service routine or the AEOI bit is set.

If the ISR ( in service ) bit is set, all the same or lower priority interrupts are inhibited but higher levels will generate an interrupt, that will be acknowledge only if the microprocessor interrupt enable flag IF is set. The priorities can afterwards be changed by programming the rotating priority modes.

### **End of Interrupt (EOI) :**

The ISR bit can be reset either with AEOI bit of ICW1 or by EOI command, issued before returning from the interrupt service routine. There are two types of EOI commands specific and non-specific. When 8259A is operated in the modes that preserve fully nested structure, it can determine which ISR bit is to be reset on EOI. When non-specific EOI command is issued to 8259A it will be automatically reset the highest ISR bit out of those already set.

When a mode that may disturb the fully nested structure is used, the 8259A is no longer able to determine the last level acknowledged. In this case a specific EOI command is issued to reset a particular ISR bit. An ISR bit that is masked by the corresponding IMR bit, will not be cleared by non-specific EOI of 8259A, if it is in special mask mode.

### **Automatic Rotation :**

This is used in the applications where all the interrupting devices are of equal priority. In this mode, an interrupt request IR level receives priority after it is served while the next device to be served gets the highest priority in sequence. Once all the device are served like this, the first device again receives highest priority.

### **Specific Rotation :**

In this mode a bottom priority level can be selected, the selected bottom priority fixes other priorities. If IR5 is selected as a bottom priority, then IR5 will have least priority and IR4 will have a next higher priority. Thus IR6 will have the highest priority.

### **Special Fully Nested Mode :**

This mode is used in more complicated system, where cascading is used and the priority has to be programmed in the master. This is somewhat similar to the normal nested mode. In this mode, when an interrupt request from a certain slave is in service, this slave can further send

## UNIT 6# Basic I/O, Memory R/W and Interrupt Operations

---

request to the master, if the requesting device connected to the slave has higher priority than the one being currently served. In this mode, the master interrupt the CPU only when the interrupting device has a higher or the same priority than the one current being served. In normal mode, other requests than the one being served are masked out. When entering the interrupt service routine the software has to check whether this is the only request from the slave. This is done by sending a non-specific EOI can be sent to the master, otherwise no EOI should be sent. This mode is important, since in the absence of this mode, the slave would interrupt the master only once and hence the priorities of the slave inputs would have been disturbed.

### **Cascade Mode :**

The 8259A can be connected in a system containing one master and eight slaves (maximum) to handle up to 64 priority levels. The master controls the slaves using CAS0-CAS2 which act as chip select inputs (encoded) for slaves. In this mode, the slave INT outputs are connected with master IR inputs. When a slave request line is activated and acknowledged, the master will enable the slave to release the vector address during second pulse of INTA sequence.

The cascade lines are normally low and contain slave address codes from the trailing edge of the first INTA pulse to the trailing edge of the second INTA pulse. Each 8259A in the system must be separately initialized and programmed to work in different modes. The EOI command must be issued twice, one for master and the other for the slave. A separate address decoder is used to activate the chip select line of each 8259A.